

RAINY DAY

Advanced Radar Tracking in Degraded Weather

Technical Reference & Handout Document

Boeing-Sponsored Senior Capstone Project

University of Washington

Team Adeptus

Daniel Trofimchik • James Gallegos • Kaz Foster • Michael Harding • Olega Obini

April 17, 2026 | Version 3.4.2

Abstract

Rainy Day is a modular MATLAB simulation framework for evaluating radar target tracking performance under real-world degraded conditions. Sponsored by Boeing as a senior capstone project at the University of Washington, the framework enables users to compose custom radar scenarios by combining independent JSON configuration files for sensors, targets, terrain, trackers, and weather; without writing any MATLAB code.

The framework models five physically motivated environmental effects: horizon masking (4/3 Earth model), ground clutter (terrain and frequency-dependent), terrain occlusion (line-of-sight via heightmaps), Doppler/MTI fade (clutter notch), and RCS-dependent detection. On top of these, a comprehensive weather degradation system supports four weather types: rain, snow, fog, and icing; each with configurable storm window profiles (step, ramp, or pulse onset).

Key capabilities include 19 sensor types (from airport PSR to airborne AESA), 10 target flight behaviors (including custom waypoints and real NASA flight data), 5 terrain presets, 3 tracker algorithms (GNN, JPDA, TOMHT) with both CV and IMM filters, a two-pass automated tracker parameter optimizer, and an interactive beam envelope visualization showing exact sensor coverage.

All features are validated by an automated test suite of 27 checks across 9 test cases. As of v3.4.2, the suite runs 27/27 green, with results reproducible via a single command: `runTestPlan`.

Contents

Abstract.....	2
1. Architecture & Modularity.....	5
1.1 Design Philosophy	5
1.2 Configuration System	5
1.3 Run File Format	5
1.4 Template System.....	6
1.5 Data Flow	6
1.6 Detection Caching.....	6
2. Environment & Physics Modeling.....	7
2.1 Horizon Masking	7
2.2 Ground Clutter	7
2.3 Terrain Occlusion.....	9
2.4 Doppler/MTI Fade	9
2.5 RCS Signatures	11
2.6 Beam Envelope Visualization.....	12
3. Weather Degradation System.....	13
3.1 Rain	13
3.2 Snow	14
3.3 Fog	14
3.4 Icing	16
3.5 Storm Window Profiles.....	17
3.6 Weather Config Example.....	17
4. Tracker Algorithms.....	18
5. NASA Real Flight Data Integration.....	19
5.1 Data Source.....	19
5.2 How It Works.....	19
5.3 Why Real Flight Data Matters	19
5.4 Demo Progression.....	19
6. Auto-Tune Tracker.....	20
6.1 Two-Pass Architecture.....	20
6.2 Composite Scoring	20
7. Validation & Test Plan.....	21
8. Code Origin & References.....	22
8.1 Custom Code.....	22

8.2 MATLAB Toolbox Dependencies	22
8.3 External References	22
9. Quick Reference Commands	23
Appendix A: What's New in v3.4.2	24

1. Architecture & Modularity

1.1 Design Philosophy

The central design goal is that a user should be able to create and run a custom radar tracking scenario in under 10 minutes, using only a text editor and the MATLAB command window. No knowledge of MATLAB programming, radar equations, or tracking algorithm internals is required.

This is achieved through a fully JSON-driven configuration system where every tunable parameter is exposed in human-readable config files with documented templates. The MATLAB code base acts as an engine that reads these configs and executes the simulation pipeline automatically.

1.2 Configuration System

The configuration system is organized into five independent component categories:

Component	Directory	Content	Examples
Sensors	config/sensors/<TYPE>/	Radar params, mounting, FOV, range, frequency	PSR, SSR, AESA, FLIR, MARITIME (19 total)
Targets	config/targets/<BEHAVIOR>/	Flight path, speed, altitude, RCS, behavior	crossing_pair, orbit, waypoints, recorded_flight
Terrain	config/terrain/<TYPE>/	Environment toggles, density, elevation	water, rural, urban, mountain, desert
Trackers	config/trackers/<ALGO>/	Algorithm params, gates, thresholds, filter	GNN, JPDA, TOMHT + tracker_globals.json
Weather	config/weather/<TYPE>/	Type, intensity, storm window, profile	rain, snow, fog, icing + weather_template

A run file (config/runs/*.json) assembles components by reference. The runSingleScenario entry point reads the run file, loads all referenced configs, builds the MATLAB trackingScenario, generates detections through the environment pipeline, runs each specified tracker, and produces metrics and 3D visualizations.

1.3 Run File Format (v3.4.2)

The degradation block consolidates all environmental toggles in one place:

```
{
  "sensors": ["PSR/default_PSR", "SSR/default_SSR"],
  "targets": "crossing_pair/default_crossing_pair",
  "terrain": "mountain/default_mountain",
  "trackers": ["GNN/default_GNN", "JPDA/default_JPDA"],
  "degradation": {
    "terrain_occlusion": true,
    "horizon_masking": true,
    "ground_clutter": true,
    "doppler_fade": true,
    "rcs_range_filter": false,
    "weather": "rain/default_rain"
  }
}
```

Weather references a file in config/weather/<TYPE>/<file>.json, or "none" for clear skies. rcs_range_filter (new in v3.4.2) is an opt-in probabilistic R^4 Pd scaler. OFF by default for user demos, ON for TC-05 validation.

1.4 Template System

Every config folder contains three standardized files for progressive user engagement:

- **default_<TYPE>.json**
 - works immediately with sensible defaults; zero customization needed.
- **<TYPE>_template.json**
 - fully documented with field descriptions, valid ranges, and comments.
- **my_<TYPE>.json**
 - user's working copy to customize freely.

1.5 Data Flow

```
runSingleScenario("my_run")
├─ loadRunFile("my_run")      → Load JSON, resolve components
├─ buildSensor()              → Create fusionRadarSensor per config
├─ addTargetFromDef()         → Build platform + trajectory per target
├─ generateTerrain()          → Procedural heightmap via groundSurface
├─ validateScenarioConfig()   → Pre-flight checks (10 categories)
├─ runDetections()            → Radar step loop + 5 env layers + weather
├─ buildTracker()             → Instantiate GNN/JPDA/TOMHT from JSON
├─ runTracker()               → Track, compute metrics, 3D visualization
└─ Save results + cache detections
```

1.6 Detection Caching

After the first run, setting "use_cached_detections": true skips detection generation entirely. The tracker runs on cached data in ~0.1–0.5 seconds instead of 30–60 seconds. This is critical for the auto-tuner and for rapid tracker comparison demos.

2. Environment & Physics Modeling

The detection pipeline applies five independent layers of physically-motivated environmental effects, plus a comprehensive weather degradation system. Each layer can be toggled on/off in the run file's degradation block.

2.1 Horizon Masking

Implementation: `isAboveHorizon.m` computes the maximum line-of-sight range using the 4/3 effective Earth radius model, which accounts for the small downward bending (refraction) of radar waves as they pass through atmospheric density gradients. By pretending the Earth is 33% larger than it really is, the geometry of an actually refracted ray can be treated as a straight line over a smoothed Earth. This is a standard engineering shortcut that is accurate for typical atmospheric conditions.

Horizon Range for a Single Antenna

$$R_h = (2 \cdot k \cdot R_E \cdot h)^{1/2}$$

Symbol	Meaning	Units
R_h	Distance to the radar horizon	m
k	Effective Earth radius factor ($k = 4/3$)	dimensionless
R_E	True Earth radius (6.371×10^6)	m
h	Height of the antenna (or target) above ground	m

Practical Form

Substituting the constants ($k = 4/3$, $R_E = 6.371 \times 10^6$ m), with h in meters and R_h in km:

$$R_h [\text{km}] \approx 4.12 \cdot h^{1/2} \quad (h \text{ in m})$$

A 30 m radar tower therefore has $R_h \approx 22.6$ km. A target flying at 10,000 m has $R_h \approx 412$ km.

Maximum Mutual Visibility Range

Because the horizon belongs to both the sensor and the target, the maximum range at which they can see each other is the sum of their individual horizon distances:

$$R_{\max} = R_h^{\text{sensor}} + R_h^{\text{target}} = (2 \cdot k \cdot R_E)^{1/2} \cdot (h_s^{1/2} + h_t^{1/2})$$

A detection is retained only if the ground range between sensor and target is less than or equal to $R_{\max}$.

Reference: Skolnik, Introduction to Radar Systems, Ch. 2; MATLAB horizonrange() documentation.

2.2 Ground Clutter (frequency-dependent, v3.4.1)

Implementation: `generateGroundClutter.m` uses a two-component model: **surface clutter** (diffuse returns from the beam to ground footprint, density falling off with range) and **discrete clutter** (strong returns from individual terrain features such as buildings, bridges, ridgelines). Both components scale with radar frequency, but via different physics.

Surface Clutter Frequency Scaling (Rayleigh)

$$N_{\text{surf}}(f) = N_{\text{ref}} \cdot (f / f_{\text{ref}})^2$$

Symbol	Meaning	Units
$N_{\text{surf}}(f)$	Expected surface-clutter returns per scan at frequency f	count
N_{ref}	Baseline count at the S-band reference frequency	count
f	Radar center frequency	Hz
f_{ref}	Reference frequency (S-band, 2.8 GHz)	Hz

The exponent of 2 comes from Rayleigh scattering: for scatterers small relative to wavelength (rain, dust, fine surface roughness), scattered power rises with f^2 . An X-band (9 GHz) radar therefore sees roughly $(9/2.8)^2 \approx 10\times$ as many clutter returns as an S-band airport radar looking at the same terrain.

Discrete Clutter Frequency Scaling (Empirical)

$$N_{\text{disc}}(f) = N_{\text{ref}} \cdot (f / f_{\text{ref}})^{0.8}$$

The lower exponent reflects measurement data: large structures (already much bigger than the wavelength) scatter more consistently across bands, meaning their returns grow with frequency but more slowly than for small scatterers.

Clutter Density vs. Range (Inverse-Square)

$$\rho(R) = \rho_0 \cdot (R_0 / R)^2$$

Symbol	Meaning	Units
$\rho(R)$	Clutter density at range R	returns/m ²
ρ_0	Reference density at calibration range R_0	returns/m ²
R	Range from radar	m
R_0	Reference range for calibration	m

Density falls with R^2 because the radar beam's ground footprint grows as R^2 , spreading the same illumination energy over a larger area.

Per-Scan Return Count (Poisson)

$$N_{\text{clutter}} \sim \text{Poisson}(\lambda_{\text{clutter}})$$

where λ_{clutter} is the expected count derived from the terrain type and frequency-scaling formulas above. The Poisson distribution captures the randomness: urban terrain may average 8 returns per scan, but any single scan could yield 4 or 12.

Terrain Presets

Terrain	Returns/Scan (S-band)	Noise (m)	Rationale
Water	~1–2	50	Flat, minimal backscatter
Rural	~3–6	100	Trees, rolling hills
Urban	~8–15	150	Building reflections
Mountain	~5–10	200	Ridgelines, peaks

Reference: Skolnik Ch. 7 (Ground Clutter); Nathanson, *Radar Design Principles*, 2nd ed. (1999).

2.3 Terrain Occlusion

Implementation: generateTerrain.m creates procedural heightmaps (200×200 grid) for 5 terrain types. The heightmap is attached via MATLAB's groundSurface() API, and SurfaceManager.occlusion() performs line-of-sight checks between every sensor-target pair at each simulation step. The radar sits on an auto-generated hilltop clearing for realistic elevated positioning.

Reference: MATLAB Mapping Toolbox: groundSurface(), SurfaceManager. Procedural generation with deterministic seed (rng 42).

2.4 Doppler/MTI Fade

Implementation: applyDopplerFade.m computes each target's radial velocity. First determining how fast it is moving directly toward or away from the radar, then projecting the target's velocity vector onto the radar-to-target line of sight. Targets moving mostly sideways (tangential) have near-zero radial velocity and look just like stationary ground clutter to an MTI filter, causing them to be suppressed.

Radial Velocity

$$v_r = \mathbf{v}_{\text{target}} \cdot \hat{\mathbf{u}}_{\text{radar} \rightarrow \text{target}}$$

Symbol	Meaning	Units
v_r	Radial velocity (component along line of sight)	m/s
$\mathbf{v}_{\text{target}}$	Target velocity vector in NED frame	m/s
$\hat{\mathbf{u}}_{\text{radar} \rightarrow \text{target}}$	Unit vector from radar to target	dimensionless

A positive v_r means the target is moving away from the radar; a negative value means it is approaching. Only this component produces a Doppler shift: the tangential component is invisible to Doppler processing.

Minimum Detectable Velocity

$$\text{MDV} = (\lambda \cdot \text{PRF}) / 4$$

Symbol	Meaning	Units
MDV	Minimum detectable velocity (clutter notch half-width)	m/s
λ	Radar wavelength ($\lambda = c / f$)	m
PRF	Pulse repetition frequency	Hz
c	Speed of light (3×10^8)	m/s
f	Radar carrier frequency	Hz

The factor of 4 in the denominator comes from the standard MTI canceller response: a two-pulse canceller has its first null at DC (zero Doppler) and its first peak at PRF/2 in Doppler frequency. The half-power point of the notch sits at roughly one-quarter of the way up, which maps to $\lambda \cdot \text{PRF}/4$ in velocity units.

Detection Probability Scaling

$$P_d(v_r) = \max(P_{\text{floor}}, P_{\text{d,nom}} \cdot |v_r| / \text{MDV}) \quad \text{when } |v_r| < \text{MDV}$$

$$P_d(v_r) = P_{\text{d,nom}} \quad \text{when } |v_r| \geq \text{MDV}$$

Symbol	Meaning	Units
$P_d(v_r)$	Effective detection probability	Dimensionless [0 to 1]
$P_{\text{d,nom}}$	Radar's baseline detection probability (from sensor config)	dimensionless
P_{floor}	Sidelobe leakage floor (set to 0.05)	dimensionless

When the target's radial velocity is at or above MDV, detection is unaffected. Below MDV, the probability ramps linearly from nominal at the boundary down toward zero at $v_r = 0$, but never falls below the 5% floor modelling the fact that real MTI filters don't achieve perfect cancellation and some signal energy always leaks through sidelobes.

Worked Examples

Band	Frequency	λ	PRF (typ.)	MDV
S-band	2.8 GHz	0.107 m	1000 Hz	≈ 27 m/s
X-band	9.0 GHz	0.033 m	1000 Hz	≈ 8 m/s

Higher-frequency radars produce tighter Doppler bins and therefore smaller MDVs. They are more sensitive to slow or tangential targets. This is one reason X-band fire-control radars can track hovering helicopters that an S-band airport radar would miss.

Reference: Skolnik, Ch. 3 (MTI and Pulse Doppler Radar); MathWorks Doppler estimation examples.

2.5 RCS Signatures (+ applyRCSFilter, v3.4.2)

Implementation: buildRCSProfile.m creates azimuth × elevation RCS pattern matrices (37×19 points at 10° steps) that fusionRadarSensor natively interpolates at each scan via platform.Signatures. Five preset profiles: stealth (−10 dBsm nose), fighter (+5 nose, +15 broadside), airliner (+20 uniform), drone (−5 base), and missile (−15 nose).

v3.4.2 adds an opt-in probabilistic RCS range filter (applyRCSFilter) layered on top of the sensor's native Swerling model. It exists because at typical scan counts (20–100 scans), the sensor-native P_d differential is too subtle to cleanly expose the R^4 dependence. Both high-RCS and low-RCS targets saturate near the configured DetectionProbability. The filter enforces the textbook radar equation explicitly.

The Radar Equation (Range Dependence)

The peak received power at the radar for a point target scales as:

$$P_r \propto (P_t \cdot G^2 \cdot \lambda^2 \cdot \sigma) / ((4\pi)^3 \cdot R^4)$$

Symbol	Meaning	Units
P_r	Received signal power at the radar	W
P_t	Transmitted power	W
G	Antenna gain (same for TX and RX)	dimensionless
λ	Radar wavelength	m
σ	Target RCS	m ²
R	Range from radar to target	m

The key takeaway for tracking is the R^4 dependence: double the range and return power drops by 2⁴. RCS differences that are invisible at short range become glaringly apparent at long range.

SNR Factor Relative to Reference

Holding all radar parameters constant and taking a ratio cancels P_t , G , λ , and constants, leaving:

$$\text{SNR}_{\text{factor}} = (\sigma / \sigma_{\text{ref}}) \cdot (R_{\text{ref}} / R)^4$$

Symbol	Meaning	Units
$\text{SNR}_{\text{factor}}$	Relative SNR vs. the reference condition	dimensionless
σ	Target RCS	m ²
σ_{ref}	Reference RCS (config parameter)	m ²
R	Target range	m

Symbol	Meaning	Units
R_{ref}	Reference range at which a σ_{ref} target is reliably detected	m

If $\sigma = \sigma_{\text{ref}}$ and $R = R_{\text{ref}}$, the factor is 1.0 (reliably detected). A $10\times$ smaller RCS at $2\times$ the range gives $\text{SNR}_{\text{factor}} = 0.1 \times (1/2)^4 = 0.00625$. A whole two orders of magnitude weaker.

Per-Detection Keep Probability

$$P_{d,\text{scale}} = \begin{cases} 1 & \text{if } \text{SNR}_{\text{factor}} \geq 1 \\ \text{SNR}_{\text{factor}} & \text{if } P_{\text{floor}} < \text{SNR}_{\text{factor}} < 1 \\ P_{\text{floor}} & \text{if } \text{SNR}_{\text{factor}} \leq P_{\text{floor}} \end{cases} \quad \begin{array}{l} : \text{ saturation} \\ : \text{ proportional} \\ : \text{ sidelobe leakage} \end{array}$$

Symbol	Meaning	Units
$P_{d,\text{scale}}$	Probability of keeping this detection	dimensionless
P_{floor}	Sidelobe-leakage floor (fixed at 0.05)	dimensionless

When the reference condition is satisfied or exceeded, the detection is always kept (saturation at 1.0). For weak-target / long-range cases, the keep-probability ramps linearly toward the 5% floor, which models sidelobe leakage and Swerling target-amplitude fluctuation: even 'impossible' detections occasionally survive due to statistical scintillation.

Reference: Skolnik Ch. 2 (Radar Equation); Knott/Shaeffer/Tuley, Radar Cross Section. R2025b note: HasRCSSignature property does not exist; the sensor reads platform.Signatures natively.

2.6 Beam Envelope Visualization

Implementation: drawBeamEnvelope.m queries coverageConfig() for exact scan limits and draws green (included) and red (excluded) cones for any radar or IR sensor. This provides immediate visual feedback on where the sensor can and cannot see, and is automatically called by plotInitialScenario.

New in v3.4.1. Replaces the previous VCP propagation model which was removed due to architectural limitations.

3. Weather Degradation System

The weather system (new in v3.4.0) provides four weather types, each with distinct physics and configurable storm window profiles. Weather configs live in `config/weather/<TYPE>/` and are referenced from the run file's degradation block.

3.1 Rain (ITU-R P.838-3)

Rain uses MATLAB's `rainpl()` function, which implements the ITU-R P.838-3 international standard for frequency-dependent specific attenuation. Three effects are modelled: range-dependent P_d reduction from two-way path loss, measurement-noise increase due to wet-radome loss, and Poisson-distributed weather clutter within the sensor FOV whose density scales as f^2 (Rayleigh).

Specific Attenuation (ITU-R P.838-3)

$$\gamma_R = k \cdot R^\alpha \quad [\text{dB/km}]$$

Symbol	Meaning	Units
γ_R	Specific attenuation (loss per unit path length)	dB/km
R	Rain rate	mm/hr
k	Frequency-dependent coefficient (from ITU tables)	dimensionless
α	Frequency-dependent exponent (from ITU tables)	dimensionless

Both k and α are tabulated by ITU-R for every radar band and polarisation. At S-band, $k \approx 0.0001$ and $\alpha \approx 1.2$, rain barely matters. At X-band, $k \approx 0.01$, a hundred-fold increase, which is why X-band radars are so sensitive to rain.

Two-Way Path Loss

The signal must traverse the rain cell twice (transmit, then echo), so the total loss doubles:

$$L_{\text{rain}} = 2 \cdot \gamma_R \cdot d_{\text{path}} \quad [\text{dB}]$$

Symbol	Meaning	Units
L_{rain}	Total two-way path loss	dB
d_{path}	Length of the signal path through rain	km

Effective Detection Probability

Path loss reduces the effective SNR and therefore the probability of detection:

$$P_{d,\text{rain}} = \max(P_{\text{floor}}, P_{d,\text{nom}} \cdot 10^{-L_{\text{rain}}/10})$$

Symbol	Meaning	Units
$P_{d,\text{rain}}$	Effective detection probability with rain	dimensionless
$P_{d,\text{nom}}$	Baseline (clear-sky) detection probability	dimensionless
P_{floor}	Configurable floor (typ. 0.15)	dimensionless

Representative Values at 16 mm/hr Rain, 100 km Path

Band	Freq.	γ_R (dB/km)	Two-Way L_{rain}	P_d Impact
S-band	2.8 GHz	~0.001	~0.2 dB	Negligible
C-band	5.6 GHz	~0.012	~2.5 dB	Moderate
X-band	9.0 GHz	~0.040	~8.0 dB	Severe

Reference: ITU-R Recommendation P.838-3; MATLAB `rainpl()` documentation.

3.2 Snow (Gunn & East, 1954)

Snow uses `rainpl()` at 25% of the equivalent precipitation rate. Gunn & East (1954) established empirically that dry snow produces roughly one-quarter the attenuation of the equivalent liquid-water precipitation rate, because snow is less dense than water and has different dielectric properties.

Equivalent Rain Rate

$$R_{eq} = 0.25 \cdot R_{snow}$$

Symbol	Meaning	Units
R_{eq}	Equivalent liquid-water rain rate	mm/hr
R_{snow}	Nominal snowfall rate (as configured)	mm/hr

Snow Attenuation (Chained Through ITU Formula)

$$\gamma_{snow} = k \cdot (0.25 \cdot R_{snow})^\alpha \quad [\text{dB/km}]$$

The path-loss and P_d equations from the Rain section then apply with γ_R replaced by γ_{snow} . Volume clutter is also reduced relative to rain, reflecting the lower backscatter cross-section of ice particles versus water droplets.

Reference: Gunn & East (1954); ITU-R P.838-3 extended for cryohydrometeors.

3.3 Fog (ITU-R P.840 / Koschmieder)

Fog is primarily a visibility-based effect that dominates IR and optical sensors, with negligible RF impact below 10 GHz. MATLAB's `fogpl()` handles RF attenuation at X-band and above.

Koschmieder Visibility Law

When you look at a distant object through fog, two things happen continuously along the line of sight: some of the light reflected from the object is scattered away by fog droplets (signal loss), and some of the surrounding ambient light is scattered *into* your line of sight along the same path (added glare). Together these cause the object's contrast against its background to fade exponentially with distance, until eventually the object becomes indistinguishable from the fog itself. The distance at which this happens is what we call *visibility*.

Harald Koschmieder (1924) made the law practical by anchoring it to human perception: he observed that the average person stops being able to distinguish an object from its background once the contrast ratio

drops to about 2%. Setting the exponential contrast decay equal to this 2% threshold gives the familiar form:

$$V = \frac{3.912}{\beta_{ext}}$$

Symbol	Meaning	Units
V	Meteorological visibility range	m
β_{ext}	Volumetric extinction coefficient	m^{-1}
3.912	Koschmieder constant $\ln\left(\frac{1}{0.02}\right)$, from 2% contrast threshold)	dimensionless

In other words, the "3.912" is not a fog constant at all. It's a *human-perception* constant that turns visibility into a physically measurable quantity. The extinction coefficient β_{ext} packs two pieces of physics into one number: how many fog droplets are crammed into each cubic meter of air, and how effectively each droplet scatters light at the sensor's wavelength. Thicker fog $\rightarrow$ more droplets $\rightarrow$ larger $\beta_{ext} \rightarrow$ shorter visibility.

Typical values:

Fog density	β_{ext}	Visibility
Light	$\approx 0.004 \text{ m}^{-1}$	$\approx 1 \text{ km}$
Moderate	$\approx 0.013 \text{ m}^{-1}$	$\approx 300 \text{ m}$
Dense	$\approx 0.04 \text{ m}^{-1}$	$\approx 100 \text{ m}$

The simulation accepts a human-friendly fog-density number (rain_rate_mmhr reinterpreted: 5 = light, 15 = moderate, 30 = dense) and uses Koschmieder to convert it into β_{ext} , which then feeds the Beer-Lambert transmittance equation $T = e^{-\beta_{ext} \cdot d}$ to compute how much of an IR or optical signal survives the path from target to sensor.

IR Transmittance (Beer-Lambert)

For IR/optical sensors, transmittance through fog over a path of length d follows Beer-Lambert:

$$T = e^{-\beta_{ext} \cdot d}$$

Symbol	Meaning	Units
T	Transmittance (fraction of signal surviving)	dimensionless
d	Path length through fog	m

The `rain_rate_mmhr` field is reinterpreted as fog density: 5 = light (~1 km visibility), 15 = moderate (~300 m), 30 = dense (~100 m). No volume clutter is generated.

Reference: ITU-R Recommendation P.840; Koschmieder (1924).

Why RF radar barely cares about fog. Fog droplets are only 1 μm to 50 μm across. S-band radar (2.8 GHz) uses wavelengths around 107 mm, thousands of times larger than the droplets themselves. So, RF waves essentially don't interact with them. Only at X-band (~33 mm) and above does fog cause measurable RF attenuation, and even there the effect is dwarfed by rain. IR sensors, by contrast, operate at wavelengths 3 μm to 14 μm , comparable in size to the droplets, making the worst possible case for scattering efficiency. Which is why fog is the *primary* visibility degrader for IR tracking.

3.4 Icing (Antenna Hardware Degradation)

Icing models ice accumulation on the radar antenna surface, not on the propagation path. This is range-independent hardware degradation; unlike rain, fog, or snow, the loss does not grow with target distance.

Gain Reduction with Severity

$$G_{\text{eff}} = G_{\text{nom}} - L_{\text{ice}}(s)$$

Symbol	Meaning	Units
G_{eff}	Effective antenna gain with ice	dB
G_{nom}	Nominal (clean-antenna) gain	dB
$L_{\text{ice}}(s)$	Severity-dependent gain loss	dB
s	Ice severity parameter	dimensionless

Two-Way Detection Impact

Because the antenna participates in both TX and RX, the total two-way loss is $2 \cdot L_{\text{ice}}$:

$$P_{\text{d,ice}} = \max(P_{\text{floor}}, P_{\text{d,nom}} \cdot 10^{-2L_{\text{ice}}/10})$$

Severity Mapping

Severity	Ice Type	L_ice	Two-Way Loss	Impact
5	Light rime	~2 dB	~4 dB	Mild detection loss
15	Moderate glaze	~4 dB	~8 dB	Noticeable range reduction
30	Severe glaze	~6 dB	~12 dB	Significant degradation

Unlike path-based weather effects, ice loss is applied uniformly regardless of target range, flatly lowering P_d for all targets in the field of view. No weather clutter is generated.

Reference: Engineering data on radome ice accretion; IEEE AESS antenna loss measurements.

3.5 Storm Window Profiles

The storm window controls when and how weather effects ramp in and out during the simulation. Three profiles are available:

- **Step:** Binary on/off. Severity = 0 outside window, 1 inside. Instant onset and clearing.
- **Ramp:** Linear ramp from 0 to 1 over first half of window, then back to 0. Gradual onset and clearing; most realistic for real storms.
- **Pulse:** Burst of severity = 1 for first 20% of window, then 0. Models a sharp initial squall that quickly passes.

Storm window timing (storm_start_s, storm_end_s) and profile type (active_type) are set in the weather config JSON. The getWeather.m function computes severity $w \in [0,1]$ at each timestep and scales the rain rate accordingly.

3.6 Weather Config Example

```
{
  "type":          "rain",
  "rain_rate_mmhr": 16,
  "storm_start_s": 100,
  "storm_end_s":   500,
  "active_type":   "ramp",
  "pd_floor":      0.15,
  "clutter_multiplier": 1.0
}
```

config/weather/rain/nasa_rain.json — Custom weather for NASA flight demo. Each validation test case uses config/weather/rain/validation_rain.json (5–50s storm window aligned to scenario duration).

4. Tracker Algorithms

Three tracking algorithms are supported, all built by the same factory function (buildTracker.m) from JSON configuration. Users switch algorithms by changing one line in their run file.

	GNN	JPDA	TOMHT
Algorithm	Global Nearest Neighbor	Joint Probabilistic Data Association	Track-Oriented Multi-Hypothesis
Assignment	Hard (1-to-1)	Soft (probabilistic)	Multiple hypotheses
Speed	Fastest	Medium	Slowest
Best For	Sparse, low-clutter	Medium clutter, real-time	Dense scenarios, highest accuracy
Motion Model	CV or IMM	CV or IMM	CV or IMM

IMM (Interacting Multiple Model) combines a Constant Velocity filter with a Coordinated Turn filter, switching between them based on measurement fit. MATLAB note: trackingIMM.TrackingFilters is read-only after construction. ProcessNoise and StateCovariance must be set before passing filters to trackingIMM().

5. NASA Real Flight Data Integration

5.1 Data Source

NASA's DASHlink portal provides Flight Data Recorder (FDR) telemetry from commercial aircraft. Each .mat file contains ARINC 717 decoded parameters at 1–16 Hz: LATP (latitude), LONP (longitude), ALT (altitude), GS (ground speed), and TRK (track angle). The Tail 687 dataset contains multiple flights covering different routes, durations, and maneuver profiles.

5.2 How It Works

- `loadNASAFlight.m` extracts the airborne trajectory from FDR data, converts GPS lat/lon to NED waypoints relative to a reference point, and resamples to uniform waypoint spacing.
- `addTargetFromDef.m` (case 'recorded_flight') feeds the NED waypoints to MATLAB's `waypointTrajectory`, creating a real flight path as a scenario target.
- Multi-target capability: Multiple real flights can be loaded simultaneously by listing them in the target config JSON, each with different RCS values (e.g., airliner 10 dBsm, regional jet 3 dBsm, small aircraft -5 dBsm).
- Globe viewer: `viewNASAFlightGlobe.m` provides an Earth-centered 3D view of real flights with simulated radar sites.

5.3 Why Real Flight Data Matters

Synthetic targets follow mathematically perfect paths. Real aircraft experience turbulence, pilot corrections, wind shear, and ATC maneuvers that stress trackers in ways synthetic data cannot. The framework bridges simulation and reality: the radar and trackers are simulated (allowing controlled experiments), but the flight path is real.

5.4 Demo Progression

The NASA flight demo shows framework flexibility by iteratively changing one variable between runs:

- Demo 1 (Baseline): Real flight, water terrain, clear weather, GNN tracker.
- Demo 2 (Add Terrain): Same flight, mountain terrain, adds occlusion, clutter, horizon masking.
- Demo 3 (Add Rain): Mountain + moderate rain (16 mm/hr ramp), weather stacks on terrain.
- Demo 4 (Heavy Rain): Mountain + 40 mm/hr, significant track degradation.
- Demo 5 (Snow): Mountain + snow, demonstrates different weather type with distinct physics.
- Tracker Swap: Using cached detections from any demo, swap GNN → JPDA → TOMHT instantly.

6. Auto-Tune Tracker

6.1 Two-Pass Architecture

Pass 1 — Tracker Parameters: Sweeps gate size, volume, beta, and algorithm-specific params. Smart grid: varies 2 params at a time + corner cases. ~60–100 combinations.

Pass 2 — Filter Parameters: Using best tracker params from Pass 1, sweeps IMM filter params: accel noise (H/V), omega dot, transition prob, init speed. Centered on baseline with 0.5×–4× variations.

6.2 Composite Scoring

Metric	Weight	Formula Term	Purpose
posRMS	0.50	$w_1 \times (\text{posRMS} / \text{maxRange})$	Position accuracy
swapCount	0.25	$w_2 \times \text{swapCount}$	Track identity stability
falseTracks	0.15	$w_3 \times \log(1 + \text{falseTracks})$	Clutter rejection
breakCount	0.10	$w_4 \times \text{breakCount}$	Track continuity

Uses cached detections exclusively. Each iteration takes ~0.1–0.5 s instead of 30–60 s for full detection generation. Saves best config as a new JSON file in `config/trackers/<TYPE>/autotuned_<TYPE>_<run>.json`.

7. Validation & Test Plan

The automated test suite executes 9 test cases containing 27 individual assertion checks. Run via: `runTestPlan`. Current status (v3.4.2): 27/27 passing.

TC	Name	What It Validates	Status
01	Template Usability	User-created config loads and runs end-to-end	✓ PASS
02	Baseline Clear	All 3 trackers produce valid metrics	✓ PASS
03	Rain S-band	S-band tracking holds under 16 mm/hr rain	✓ PASS
04	Rain X-band	X-band fewer target dets than S-band at same rain rate	✓ PASS
05	RCS Verification	20 dBsm vs -10 dBsm detection ratio at 100 km	✓ PASS (3.43×)
06	Crossing Swap	JPDA swap count $\leq$ GNN swap count	✓ PASS
07	Compound Stress	TOMHT + rain + mountain + mixed RCS completes	✓ PASS
08	Error Paths	5 malformed configs produce clear error messages	✓ PASS
09	Verification Suite	verifySimulation.m (40+ checks) completes	✓ PASS

TC-03 vs TC-04 use symmetric PSR sensors (sband_PSR / xband_PSR, both 111 km rangeLimits) and matched rain config (validation_rain, 5–50s storm window). The only test variable is CenterFrequency, isolating ITU-R P.838-3 attenuation. TC-05 enables the probabilistic rcs_range_filter to expose the R^4 dependence; airliner = 24 dets, stealth = 7 dets, ratio 3.43×.

8. Code Origin & References

8.1 Custom Code

- Complete JSON configuration architecture: loadRunFile.m, buildModularConfig.m, all templates
- Sensor factory (buildSensor.m): Maps 19 sensor type strings to fusionRadarSensor configurations
- Detection pipeline (runDetections.m): Orchestrates 5-layer environment model + weather system
- Environment layers: isAboveHorizon, generateGroundClutter, generateTerrain, applyDopplerFade, applyRCSFilter (v3.4.2 opt-in), buildRCSProfile, drawBeamEnvelope
- Weather system: applyWeatherDegradation (4 types), getWeather (storm window), weather configs
- Tracker factory and runner: buildTracker.m, runTracker.m, initCVFilter.m, initIMMFilter.m
- Analysis and reporting: analyzeTrackSwaps, plotInitialScenario, drawSensorCoverage, tabbedAxes, all visualization
- Auto-tuner (autoTuneTracker.m): Two-pass parameter sweep with composite scoring
- NASA flight integration: loadNASAFlight, runNASAFlight, scanNASAFlights, viewNASAFlightGlobe
- Validation suite: runTestPlan (27 checks), verifySimulation (40+ checks), 12 validation run files
- Diagnostic tools: diagBeamLimits, diagnoseBadDetections, diagVerifyTargetIndex

8.2 MATLAB Toolbox Dependencies

- Sensor Fusion and Tracking Toolbox (R2024a+): trackingScenario, fusionRadarSensor, trackerGNN/JPDA/TOMHT, trackingIMM
- Radar Toolbox: horizonrange, rainpl(), landroughness, earthSurfacePermittivity
- Mapping Toolbox: groundSurface, SurfaceManager (terrain occlusion)
- Phased Array System Toolbox: rainpl() for ITU-R P.838-3 (optional fallback available)

8.3 External References

- ITU-R P.838-3: Specific attenuation model for rain (international standard)
- ITU-R P.840: Attenuation due to fog and clouds
- Skolnik, Introduction to Radar Systems, 3rd ed.: Horizon, MTI/Doppler, clutter fundamentals
- Gunn & East (1954): Snow-to-rain equivalent attenuation factor
- Knott/Shaeffer/Tuley, Radar Cross Section: Aspect-dependent RCS profiles
- FAA ASR-11 specifications: S-band PSR baseline parameters
- NASA DASHlink (c3.ndc.nasa.gov/dashlink/resources/664/): Real flight data

9. Quick Reference Commands

All commands assume you have cd'd to the adeptus-rainyday-tracking folder and run:

```
addpath("scripts"); addpath(genpath("src"));
```

Command	What It Does
runSingleScenario("my_run")	Run a simulation from a run file
runTestPlan	Execute all 9 test cases (27 checks)
autoTuneTracker("run", "GNN")	Sweep GNN parameters on cached detections
runNASAFlight	Track real aircraft with simulated radar
viewNASAFlightGlobe	Earth-centered 3D view of real flights
scanNASAFlights	Profile all flight data files in a folder
verifySimulation	Run 40+ diagnostic checks across 8 phases
buildModularConfig	First-time setup (creates config/ folders)
recordDemoVideo	Export 1080p MP4 demo video

Appendix A: What's New in v3.4.2

v3.4.2 (April 16, 2026) is a validation pass that restored the test plan to a clean 27/27 after regressions in the v3.4.1 session. The following changes were made:

RCS Detection Architecture

- `applyRCSFilter.m` restored to `src/+trackbench/+environment/` as an opt-in probabilistic filter. Applies $Pd_scale = \max(0.05, \min(1, (\sigma/\sigma_ref) \cdot (R_ref/R)^4))$ per detection; default OFF to preserve the sensor-native Swerling model for user scenarios.
- `runDetections.m` now reads `envConfig.rcs_range_filter` (default false) and calls `applyRCSFilter` when true. A diagnostic line reports the setting at the start of each run.
- `loadRunFile.m` propagates `rcs_range_filter` from the run file's degradation block into `config.environment`.

TC-03 / TC-04 Sensor Geometry

- New `sband_PSR.json`: symmetric S-band counterpart to `xband_PSR.json` (2.8 GHz, 111 km rangeLimits). TC-03 now uses this instead of default_PSR (which has a 20 km range cap for close-in demos).
- New `validation_rain.json`: rain profile with storm window (5–50 s) aligned to the 50 s gentle_turn validation scenario. The default_rain profile's 50–130 s window never fired during validation, leaving TC-03/TC-04 effectively clear-sky. Both tests now reference `validation_rain`.
- With symmetric sensors and matched rain, the only variable between TC-03 and TC-04 is `CenterFrequency`, cleanly isolating ITU-R P.838-3 attenuation as the test subject.

TC-05 Run File Adjustments

- Sets `"rcs_range_filter"`: true in the degradation block to enable the opt-in filter for this test.
- Sets `"doppler_fade"`: false targets in TC-05 fly tangentially to the radar; Doppler fade would cull both targets equally and confound the RCS test.

Results

27/27 tests passing. TC-04 shows X-band = 6 target dets vs S-band = 14 at matched 16 mm/hr rain. TC-05 shows airliner = 24, stealth = 7, ratio 3.43×. Both numbers are within expected statistical spread for their underlying models.

Glossary of Terms

Rainy Day v3.4.2

Updated April 17, 2026

Plain-English definitions of radar and simulation terminology used throughout the Rainy Day handout and the Boeing presentation. Terms are listed alphabetically.

Term	Definition
4/3 Effective Earth Radius	A standard engineering model that accounts for how the atmosphere bends (refracts) radar waves slightly downward, letting them follow the Earth's curve farther than a straight line would. The "4/3" means we pretend the Earth is 33% larger than it really is, which simplifies the math while accurately predicting how far a radar can see.
applyRCSFilter (v3.4.2)	An optional detection filter (v3.4.2) that enforces the textbook R^4 radar equation on top of fusionRadarSensor. It exists because the sensor's built-in statistical model produces nearly identical detection rates for a big airliner and a small stealth drone over short simulation runs (20–100 scans). Both saturate near the configured DetectionProbability, hiding the expected range/RCS relationship. When enabled, each detection is kept with a probability derived from the target's RCS and range: reference-sized targets at reference range always keep, while small targets at long range drop toward a 5% floor. Default OFF; ON for TC-05 validation via "rcs_range_filter": true. Located at src/+trackbench/+environment/applyRCSFilter.m.
Atmospheric Refraction	The bending of radar waves as they pass through layers of atmosphere with different densities. Under normal conditions, this bending extends the radar's horizon beyond the geometric line-of-sight. Abnormal refraction (ducting) can extend or reduce range dramatically.
Beer-Lambert Law	The exponential-decay rule that describes how light (or any electromagnetic wave) loses intensity while passing through an absorbing or scattering medium: $T = e^{-\beta \cdot d}$, where T is transmittance (fraction of signal surviving), β is the extinction coefficient (m^{-1}), and d is path length (m). Used in §3 Fog to convert fog density (via Koschmieder) into IR signal loss along a sensor-to-target path. Not specific to fog; the same law governs atmospheric absorption, underwater light transmission, and X-ray imaging.
Clutter Notch (MTI)	A blind zone in a radar's Doppler filter where targets become invisible. MTI radars suppress stationary ground clutter by filtering out returns with zero Doppler shift. The side effect: any moving target that happens to have near-zero radial velocity (flying sideways relative to the radar) gets filtered out too, because it looks just like ground clutter.

Term	Definition
CT (Coordinated Turn)	A motion model used inside trackers that assumes the target is flying a constant-radius turn at constant speed (banking like a real aircraft). Better than Constant Velocity for maneuvering targets. In the framework, CT is never used alone; it's paired with CV inside an IMM filter so the tracker can smoothly switch between straight-line and turning behavior as measurements come in.
CV (Constant Velocity)	A motion model used inside trackers that assumes the target is flying in a straight line at constant speed. Fast, simple, and accurate for aircraft cruising between waypoints. Breaks down during maneuvers (turns, climbs, accelerations), which is why the framework pairs CV with a Coordinated Turn (CT) model inside an IMM filter for the best of both worlds.
dBsm (Decibels relative to a Square Meter)	The unit for Radar Cross Section. A 747 airliner is about +20 dBsm (large, easy to detect). A stealth aircraft might be -10 dBsm (small radar return, hard to detect). Every +10 dB means the target appears 10× larger to the radar.
Discrete Clutter	False radar returns from individual large objects on the ground, such as buildings, bridges, water towers, or parked aircraft. Unlike surface clutter (which is diffuse), discrete clutter looks like real targets because the returns are strong and localized.
Doppler Shift	The change in frequency of a radar return caused by the target's motion. Targets moving toward the radar compress the signal (higher frequency); targets moving away stretch it (lower frequency). Same effect that makes an ambulance siren change pitch as it passes you.
FOV (Field of View)	The angular region a sensor can see at any given moment. For a rotating PSR, the horizontal FOV is the beamwidth (~1.4°) but it sweeps 360° over one rotation. For a sector scanner like PAR, the FOV is the fixed angular sector it covers.
freq² Rayleigh Scattering	A physics relationship describing how small particles (raindrops, dust) scatter radar energy. The amount of scattering increases with the square of the frequency, meaning X-band (9 GHz) radar sees roughly 10× more clutter from rain than S-band (2.8 GHz) radar. Named after Lord Rayleigh who derived the relationship in the 1870s.
freq^{0.8} Scaling (Discrete Clutter)	An empirical relationship for how discrete clutter (buildings, bridges, large structures) reflects radar energy at different frequencies. The exponent 0.8 is less than the 2.0 of Rayleigh scattering because these objects are already far larger than the radar wavelength; they're in the "optical" scattering regime, where return strength depends more on geometry than on wavelength. Compare: rain droplets (small vs. wavelength) follow freq ² , but a building reflects X-band only modestly more than S-band. Exponent value comes from measured data in standard radar engineering literature.

Term	Definition
fusionRadarSensor	MATLAB's built-in radar sensor object from the Sensor Fusion & Tracking Toolbox. It models a complete radar system (antenna pattern, detection probability, range resolution, scan behavior) at a statistical level. buildSensor.m configures these objects from JSON files. In R2025b the sensor natively reads platform.Signatures (no HasRCSSignature property needed).
GNN (Global Nearest Neighbor)	A tracker algorithm that assigns each detection to exactly one track, using the single best distance match across all tracks simultaneously (the "global" part). Fast and deterministic. Best suited to sparse, low-clutter scenarios. Breaks down when two targets cross paths because it can confidently assign detections to the wrong track (see Track Swap). Fastest of the three trackers the framework supports.
Gunn & East (1954)	A foundational paper (K. L. S. Gunn and T. W. R. East, "The microwave properties of precipitation particles," Quarterly Journal of the Royal Meteorological Society, 1954) that empirically measured RF attenuation of snow vs. rain. Their finding, that dry snow attenuates radar about 25% as much as the equivalent liquid-water precipitation rate, is still the basis for the snow model in §3 Snow.
IMM (Interacting Multiple Model)	A filter that runs two motion models simultaneously (Constant Velocity for straight lines and Coordinated Turn for banking/curving) and blends their predictions based on which one best matches the incoming measurements. When a plane is flying straight, the CV model dominates. When it turns, the CT model takes over. The transition is automatic.
Inverse-Square Density	A mathematical model where clutter density decreases with the square of the distance from the radar. Close to the radar, ground clutter is dense (many false returns per scan). Far away, it thins out. This matches reality: the radar beam footprint on the ground grows with range, spreading the energy over a larger area.
ITU-R P.838-3	An international standard published by the International Telecommunication Union that defines exactly how much radio-frequency energy rain absorbs at different frequencies and rain rates. It's the global reference used by radar and telecom engineers everywhere. The formula is: $\text{attenuation} = k \times R^\alpha \text{ dB/km}$, where R is rain rate and k, α depend on frequency. MATLAB exposes it via rainpl().
JPDA (Joint Probabilistic Data Association)	A tracker algorithm that computes the probability that each detection comes from each track, then updates each track with a weighted combination of all nearby detections. Softer than GNN's hard assignment and much more robust to clutter and crossing targets, at modest computational cost. The framework's default for medium-clutter scenarios.
Koschmieder Visibility Law	The equation $V = 3.912 / \beta_{\text{ext}}$ that converts between the two common ways of describing fog: visibility range V (what ATIS reports and what

Term	Definition
	humans intuit) and extinction coefficient β_{ext} (what Beer-Lambert needs for math). The 3.912 factor equals $\ln(1/0.02)$, arising from Harald Koschmieder's 1924 observation that the human eye can barely distinguish an object from its background once contrast drops to about 2%. So the "3.912" isn't a fog constant at all; it's a human-perception constant baked into the definition of visibility. See §3 Fog for the full derivation.
LOS (Line of Sight)	A straight line between the radar antenna and a target. If terrain (mountains, buildings) blocks this line, the target cannot be detected regardless of radar power. Our simulation checks LOS for every sensor-target pair at every timestep using MATLAB's <code>SurfaceManager.occlusion()</code> .
MDV (Minimum Detectable Velocity)	The slowest radial speed a target can have and still be seen by an MTI radar. Targets moving slower than the MDV are filtered out as clutter. Calculated as $MDV = \lambda \times PRF / 4$, where λ is the radar wavelength and PRF is the pulse repetition frequency. For S-band (2.8 GHz): $MDV \approx 27$ m/s. For X-band (9 GHz): $MDV \approx 8$ m/s. Fixed in v3.4.1 to use this formula per-sensor rather than a hardcoded value.
Multipath	When a radar signal bounces off the ground (or water) before reaching the target, creating a second path alongside the direct signal. These two signals interfere with each other, sometimes adding (stronger detection) and sometimes canceling (blind spots). This creates a lobing pattern in the radar's vertical coverage.
MTI (Moving Target Indication)	A radar signal processing technique that compares successive radar pulses to detect changes (i.e., moving targets) while suppressing stationary ground clutter. It works by exploiting Doppler shift: stationary objects return the same signal each pulse, while moving targets produce slightly different returns.
NED (North-East-Down)	A coordinate system used in aerospace. North is +X, East is +Y, Down is +Z. Altitude is negative Z (higher = more negative). This is MATLAB's default coordinate system for trackingScenario.
Pd (Detection Probability)	The chance that the radar detects a target on any given scan. $Pd = 0.9$ means 90% of the time the radar sees the target. Pd depends on range (farther = lower), RCS (smaller = lower), weather (rain = lower), and many other factors.
Pd_scale (v3.4.2)	The probability that <code>applyRCSFilter</code> will keep a given detection, expressed as a number between 0.05 (5%, the floor) and 1.0 (always kept). For a target at or above the reference RCS sitting within the reference range, Pd_scale hits 1.0, and the detection is never dropped. For a small-RCS target far beyond the reference range, Pd_scale drops toward 0.05, meaning roughly 1 in 20 detections will still survive. Formula: $Pd_scale = \max(0.05, \min(1, SNR_factor))$. The 5% floor models

Term	Definition
	sidelobe leakage and Swerling RCS fluctuation; real radars never perfectly reject a target.
P_{fa} (False Alarm Rate)	The probability that the radar reports a target when nothing is there. Essentially, it's how often noise or clutter tricks the system. A Pfa of 10^{-6} means one false alarm per million resolution cells. Lower is better but requires stronger signals, reducing range.
Poisson Count	A statistical model for counting random events. In our clutter model, the number of false returns per scan follows a Poisson distribution. You can predict the average count (e.g., 5 clutter returns in urban terrain) but the exact number varies randomly each scan. This is the standard way radar engineers model clutter occurrence.
PRF (Pulse Repetition Frequency)	How many radar pulses are transmitted per second. Higher PRF means better Doppler resolution (smaller MDV) but shorter unambiguous range. A typical S-band airport radar might use PRF around 1000 Hz.
R^4 Radar Equation	The fundamental law of radar: received signal power drops as the fourth power of range. Double the distance and the return signal is $16\times$ weaker. This is why RCS differences (e.g., 20 dBsm vs -10 dBsm) only become apparent at longer ranges. At short range, even small targets return enough signal.
Radome	The weatherproof enclosure (usually a dome or cone) covering a radar antenna. Rain, ice, or dirt on the radome absorbs some radar energy, reducing performance. "Wet radome loss" is typically 0.5–1.5 dB in moderate rain.
RCS (Radar Cross Section)	A measure of how "visible" a target is to radar, expressed in square meters or dBsm. A Boeing 747 has an RCS of about 100 m ² (20 dBsm). A stealth aircraft might be 0.1 m ² (-10 dBsm). RCS depends on the target's size, shape, materials, and the angle the radar is looking at it.
rcs_range_filter (v3.4.2)	The run-file flag (inside the degradation block) that enables applyRCSFilter. Default false. When true, each detection is kept with probability $Pd_scale = \max(0.05, \min(1, SNR_factor))$ where $SNR_factor = (\sigma/\sigma_ref) \times (R_ref/R)^4$. Used by TC-05 validation to expose the R^4 RCS dependence that the sensor-native Swerling model leaves too subtle to observe at typical scan counts.
S-band / X-band / C-band	Frequency bands used by different radar types. S-band (2–4 GHz): used by airport and weather radars, penetrates rain well. C-band (4–8 GHz): compromise between resolution and weather penetration. X-band (8–12 GHz): higher resolution but severely affected by rain. Higher frequency = shorter wavelength = better resolution but worse weather performance.
Sidelobe Leakage	Even when a target is in the radar's Doppler blind zone (clutter notch) or the RCS filter's cutoff region, a small amount of signal energy "leaks"

Term	Definition
	through due to imperfect filtering. Modeled as a 5% detection floor, meaning even a worst-case tangential or low-RCS target still has a 5% chance of being detected on any given scan. Used consistently by applyDopplerFade and applyRCSFilter.
SNR (Signal-to-Noise Ratio)	The strength of the target's radar return compared to background noise. Higher SNR means more reliable detection. Rain reduces SNR by absorbing signal energy and adding noise. SNR is measured in decibels (dB).
<i>SNR_{factor}</i> (v3.4.2)	A dimensionless "how easy is this target to detect?" score used by applyRCSFilter. Computed as $SNR_factor = (\sigma_target / \sigma_ref) \times (R_ref / R_actual)^4$, it combines how big the target is (RCS ratio) with how close it is (range ratio, to the 4th power). A score of 1 means the target matches the reference detection threshold exactly; above 1 means easier to detect, below 1 means harder. Worked examples: a 20 dBsm airliner ($\sigma=100$) at 100 km with refRange=111 km, refRCS=0 dBsm scores ≈ 152 (always kept); a -10 dBsm stealth UAV ($\sigma=0.1$) at 105 km scores ≈ 0.13 (~13% kept per scan).
Storm Window	The time period during a simulation when weather effects are active. Defined by a start time, end time, and profile shape (step = instant on/off, ramp = gradual onset/clearing, pulse = brief intense burst). Outside the storm window, the weather severity $w = 0$. validation_rain.json uses a 5–50 s step window aligned to the 50 s gentle_turn validation scenario.
Swerling Model	A set of statistical models that capture how a target's radar return "flickers" from scan to scan. A real aircraft's RCS bounces around as its aspect angle changes, wings catch the beam differently, and multipath reflections add and cancel. It never returns exactly the same echo twice. Peter Swerling (1950s) classified these fluctuations into four cases. Swerling 0 is the idealized non-flickering target (same RCS every scan). Swerling 1 models a target whose RCS stays constant within one radar "look" but changes randomly between looks (e.g., a slow-aspect fighter). Swerling 3 is similar but follows a distribution better suited to large targets dominated by a single strong scatterer (e.g., airliner fuselage). Set via rcsSignature.FluctuationModel; fusionRadarSensor uses it internally to convert signal-to-noise ratio into detection probability.
TOMHT (Track-Oriented Multi-Hypothesis)	A tracker algorithm that maintains multiple hypotheses about how detections could be assigned to tracks, delaying hard decisions until enough evidence accumulates to resolve ambiguity. The most accurate of the three trackers the framework supports, but also the slowest. Use for dense scenarios where tracks must not be lost even through heavy clutter or target crossings.
Track Swap	When a tracker accidentally switches which detection belongs to which target. If two aircraft cross paths, the tracker may assign Aircraft A's future detections to Track B and vice versa. This is the primary weakness

Term	Definition
	of simple (GNN) trackers and the main advantage of probabilistic (JPDA) or multi-hypothesis (TOMHT) trackers.

Team Adeptus | University of Washington | v3.4.2